Research artifacts that people and tools can check

The Universal Artifact SDK and the artifact-judge-reviewer/v1 specification

An open format that links every paper claim to its evidence, a deterministic judge that checks those links, and a review assistant that quotes the paper. The human reviewer makes every decision.

`github.com/microsoft/universal-artifact-sdk`

Public drafts: `evaluable-artifact/v3` · `artifact-judge-reviewer/v1`

Checking one number from a paper is still detective work

> “…achieving a low error rate of 0.0011 bits per byte in converting ABC notation to MIDI format.”
>
> Wu et al., “Beyond Language Models: Byte Models are Digital World Simulators” (bGPT), abstract, page 1

To check it, a reviewer must find:

- the experiment that produced 0.0011
- the file and cell that hold it
- the code, data, and environment that ran it
- whether a re-run still gives 0.0011

The authors knew every answer when they wrote the paper. The links were lost when the work was flattened into prose and a folder of files.

Agents reproduce published results in only about 20–30% of cases (CORE-Bench 2024, REPRO-Bench 2025).

Quotes: Wu et al. 2024, arXiv:2402.19155, CC BY 4.0

An evaluable artifact links each claim to its evidence and to the check that settles it

Claim C1
“Posterior contraction improves with more feedback rounds.”

→

Result R1
`kl_initial, kl_final`

→

Evidence
`contraction.csv`

Validator
`monotonic, direction: decreasing`

Claim	a statement the paper makes
Result	a measured output, with a locator to the exact row and column
Evidence	the file that holds the result
Validator	the check that turns evidence into a verdict

Example: worked example in the SDK specification (SPEC.md, Appendix A)

Authors declare these links with a few typed calls, and the SDK writes the files

```
addResult(a, {
  id: "R1", validates: ["C1"],
  produced_by: "posterior_contraction",
  evidence: "experiments/…/contraction.csv",
  locators: { kl_final:
    { column: "kl_divergence", row: "final" } },
});
addClaim(a, {
  id: "C1",
  statement: "Posterior contraction improves…",
  validator: { kind: "builtin", name: "monotonic",
    input: { result: "R1",
             series: ["kl_initial", "kl_final"] },
    params: { direction: "decreasing" } },
});
writeSubmission(a, "./out");
```

```
out/
  manifest.yml
  claims.yml
  results.yml
  datasets.yml
  SHA256SUMS
  .sdk/state.json
```

- TypeScript and Python bindings, tested against one shared corpus
- Checks structure only: every ID resolves, every path stays inside
- Re-running rewrites the same bytes and keeps your own files

The same format covers interviews, proofs, figures, and the contributions authors claim

```
addClaim(a, { id: "C7",
  statement: "Participants distrusted
              the tool's suggestions.",
  validators: [
    { id: "v_attest", kind: "attest",
      checks: "provenance",
      inputs: ["R2"] },
    { id: "v_inspect",
      kind: "llm_judge",
      gated_by: "v_attest",
      criteria: "The cited excerpts
        express distrust…",
      inputs: ["R2"] } ] });
```

- **R2** points at interview transcripts, with Cohen’s κ = 0.81 and an ethics block
- **Attest gates inspect:** the excerpts’ provenance is checked before anyone weighs what they say
- **Exhibits** link claims to a figure, table, proof, derivation, or listing
- **Contributions** record the advances authors claim; a human judges their novelty

> Whatever its kind, every claim resolves to evidence a reader can locate.

Each claim is checked as strongly as its evidence allows, and the verdict says how

`re-executed`	the evaluator re-ran the pinned code; the check passed
`re-analyzed`	the evaluator recomputed the numbers from released data
`evidence-supported`	the check passed on the authors' shipped outputs
`attested`	the evidence is intact and its source vouches for it
`unverifiable`	declared, but cannot be settled with what is shipped

`failed` a check ran and its condition was false

`unassessable` the evidence was never declared: a coverage note

Authors declare a validation mode: **re-execute**, **re-analyze**, **inspect**, or **attest**. Only the evaluator's own work moves a claim up the ladder.

From a bare PDF, the paper adapter extracts each reported number and caps it at “the paper says so”

results.yml

```yaml
- id: R1
  validates: [C1]
  validation_mode: attest
  evidence: paper.pdf
  reported_value: 0.0011
  locators:
    paper:
      quote: "0.0011"
      page: 1
      section: abstract
```

claims.yml

```yaml
- id: C1
  statement: "…a low error rate of
    0.0011 bits per byte in
    converting ABC notation
    to MIDI format."
  validators:
  - id: v1
    kind: attest
    checks: provenance
    inputs: [R1]
```

- bGPT paper: 68 results and 2 headline claims; 903 numbers skipped (years, labels, bare integers)
- Messy tables work: “76.39±0.00” in a table of *Loss-Free Machine Unlearning* becomes one result, value 76.39

Quotes: Wu et al. 2024, arXiv:2402.19155, CC BY 4.0 · Foster et al. 2024, arXiv:2402.19308, CC BY 4.0

On the bGPT paper, the judge traces both headline numbers to the paper and rates the run PARTIAL


```
{
  "id": "claim_validators/C1.v1",
  "verdict": "advisory",
  "validation_mode": "attest",
  "mode_verdict": "attested",
  "method_badge": "attested",
  "evidence": ["claims.yml#C1",
               "results.yml#R1"],
  "grounded_in": "S1, S3"
}
```


Rubric: PARTIAL

Available, Functional, Results

Reproduced: all `unverifiable`

- No finding failed
- No model involved: the same input gives the same findings
- Every finding cites its standards and how to re-derive it

> A paper alone shows what was reported. The cap keeps the verdict at exactly that.

Verdicts roll up to three outcomes that follow ACM’s “Results Reproduced” badge

`NOT_REPRODUCED`	any check failed
`REPRODUCED`	the evaluator re-executed or re-analyzed every assessed claim; a badge names the method
`PARTIAL`	everything else, e.g. a claim settled by inspection or attestation

ACM badge	Awarded when
Available	a license file and a resolvable paper
Functional	dependencies and experiments declared; no structural failure
Results Reproduced	the rubric is REPRODUCED

Without re-running anything, the ceiling is **PARTIAL**. “Reusable” needs human judgment and stays with reviewers.

The reviewer drafts points that quote the paper, and leaves every decision to the human

Every draft begins with:

Human-owned assessment

```
Overall recommendation: unset
Criterion scores: unset
Reviewer confidence: unset
```

- A critic drops any quote that does not match the paper
- Any score or recommendation in the text is rejected
- No grounding: the point is marked unassessed

Question from the example review

Is the 0.0011 BPB for ABC→MIDI explained by MIDI bytes that are near-deterministic given the ABC?

`paper:p.8` *“BPBmidi evaluates data conversion, considering the complete ABC byte sequence is given.”*

Requested response: report the exact-match or valid-MIDI rate for greedy end-to-end conversion.

Every run is sealed: hashes bind what was read to what was written

```
{ "format_version":
    "artifact-review-run/v1",
  "status": "completed",
  "inputs": {
    "snapshot/paper/paper.pdf":
      "999e3d81…" },
  "stages": {
    "artifact_checks": "completed",
    "paper_review": "not-requested",
    "reproduction": "not-requested" },
  "outputs": {
    "judge.json": "<sha256>",
    "review-draft.md": "<sha256>" } }
```

- The input artifact is never modified, and a run is never overwritten
- Every stage reports its status: completed, unavailable, not-run, or not-requested
- Anyone can re-hash a run and verify it without trusting the tool
- A human-in-the-loop layer reads the run and never edits it

Every check cites a published standard, so verdicts carry meanings reviewers already know

ID	Standard	Grounds
S1	ACM Artifact Review and Badging v1.1	verdicts and badges
S2	NISO RP-31-2021 Reproducibility Badging	reproducible / replicable terms
S3	ML Reproducibility Checklist (Pineau et al.)	what an artifact must ship
S4–S5	NeurIPS and ACL Rolling Review forms	review criteria
S6–S7	SIGPLAN review policy and empirical checklist	claims backed by evidence; negative results

> Each finding carries `"grounded_in": "S1, S3"`. A check that no standard supports goes into an optional extension.

The specification is written so a new team can rebuild the tool from it alone

Level	Scenarios
L1 judge core	J1–J50
L2 paper adapter	A1–A35
L3 reviewer	R1–R28, scripted model
L4 reproduction	P1–P6
Real papers	E1–E5: two CC BY papers

Blind-rebuild criterion

Given only the spec and the JSON Schema, a builder passes every scenario for the levels they claim. Each gap found is a spec defect.

Example: E1

Adapter on the bGPT PDF yields `0.0011` and `99.99%`, each located on page 1, in the abstract.

Real papers are pinned by SHA-256 and downloaded at test time.

Authors record evidence once, reviewers start from checked findings, and builders share one contract

Paper authors

- Record claims, results, and evidence while you work
- Start with three: experiments, claim→evidence links, the paper
- Every field is optional; gaps become coverage notes
- Refuted hypotheses count as honest negative results

Reviewers and AECs

- Start from deterministic findings, each with a way to re-derive it
- See how each claim was checked, on one ladder
- Use a draft where every point quotes the paper
- You set scores, badges, and the recommendation

Tool builders, venues

- Build to an open schema and spec; claim levels L1–L4
- Plug in your own model, runner, and venue profile
- Read sealed runs from CI and dashboards
- Map venue badge checklists onto the profile

Today the format and specs are public drafts; automated re-execution is specified and still to come

Available now (draft)

- SDK in TypeScript and Python, release candidate 0.9
- evaluable-artifact/v3 JSON Schema
- artifact-judge-reviewer/v1 spec with conformance scenarios

In progress

- Reference judge, paper adapter, and reviewer, converging on the spec
- Reproduction provider: specified; implementation still to come
- Proposed validators: set ranking, distribution comparison

> Until a reproduction provider exists, the honest ceiling for any run is **PARTIAL**.

SDK CHANGELOG and README; artifact-judge-reviewer/v1 §4.9, §7.2

Read the spec, try the SDK on your next artifact, and tell us where it falls short

Repository	`github.com/microsoft/universal-artifact-sdk`
Why	`MOTIVATION.md`
Format spec	`SPEC.md`
Judge/reviewer spec	`docs/judge-reviewer-spec/`
Try it	`node examples/quickstart.mjs`

> Open an issue for format gaps, unclear rules, or what your venue or AEC needs. Rebuilding the judge from the spec is the most useful test of all.